DATA STRUCTURES

Unit-wise Question Collection

Extracted questions only from the five supplied PDFs

UNIT 1 – Introduction to Data Structures

Q.1 Define / One line / List (1 Mark)

1. Define Data Structure.

2. Define Abstract Data Type (ADT).

3. List the examples of Linear Data Structures and Non-Linear Data Structures.

4. Define Traversing, Insertion, and Deletion.

Q.2 Explain (2–3 Marks)

1. Explain the concept of Data Structure with a suitable example.

2. Explain the need for Data Structures and discuss any four reasons for using them.

3. Explain Abstract Data Type (ADT) with the example of Stack ADT.

4. Explain Linear and Non-Linear Data Structures with suitable examples.

5. Explain Graph and Tree Data Structures in short.

Q.3 Apply (3–4 Marks)

1. Given the array [10, 20, 30, 40, 50], demonstrate the traversal operation and state its time complexity.

2. Demonstrate the insertion of 25 between 20 and 30 in the array [10, 20, 30, 40]. Show the array after insertion.

3. Given the operations Push(10), Push(20), and Push(30), demonstrate the resulting Stack and identify the element removed by Pop().

Q.4 Analyse / Differentiate (5 Marks)

1. Differentiate between Abstract Data Type (ADT) and Data Structure based on their characteristics and implementation.

2. Differentiate between Linear and Non-Linear Data Structures with suitable examples.

Q.5 (5 Marks)

A student marks management system requires storing and processing marks of a large number of students. Evaluate the suitability of an Array or Linked List for this application, select the appropriate Data Structure, and write a C program to demonstrate its use. Justify your choice.

UNIT 2 – Searching and Sorting

Q1. Answer the following

1. Define Pivot in Quick Sort.

2. What is Binary Search?

3. List any five sorting techniques covered in the syllabus.

4. Define Searching.

Q2. Answer the following

1. Explain the difference between Linear Search and Binary Search.

2. Explain why Binary Search requires a sorted array.

3. Explain the working principle of Selection Sort.

4. Explain the working principle of Bubble Sort.

5. Explain the Divide and Conquer approach used in Merge Sort.

Q3. Solve / Apply the following

1. Apply Linear Search to find 40 in the array. Also, write a C program to implement Linear Search.
Array: 20, 15, 40, 10, 30.

2. Apply Binary Search to find 60 in the array. Also, write a C program to implement Binary Search.
Array: 10, 20, 30, 40, 50, 60, 70.

3. Apply Bubble Sort to sort the following array in ascending order. Also, write a C program to implement Bubble Sort.
Array: 64, 25, 12, 22, 11.

4. Apply Insertion Sort to sort the following array. Also, write a C program to implement Insertion Sort.
Array: 5, 3, 4, 1, 2.

5. Apply Selection Sort to the following array in ascending order. Write a C program to implement Selection Sort.
Array: 50, 30, 70, 10, 40.

Q4. Analyse the following

1. Analyze the difference between Linear Search and Binary Search with respect to: data requirement, searching method, best-case complexity, worst-case complexity, and suitable application.

2. Analyze Merge Sort and Quick Sort as divide-and-conquer algorithms. Compare their division/partitioning process, recursive processing, best-case complexity, worst-case complexity, and extra space requirement.

3. Analyze the following Selection Sort array and identify the minimum element selected in each pass:
64, 25, 12, 22, 11.

UNIT 3 – Linked List

1. In a singly linked list, the link field of the last node generally contains: (1 Mark)

   A) Address of the first node

   B) Address of the previous node

   C) NULL

   D) Address of the root node

2. Analyze the role of Node, Data field, Next pointer, and NULL pointer in the representation of a singly linked list. Explain what happens if the Next pointer of a node is incorrectly assigned (5 Marks).

3. Write a C program to implement a singly linked list with functions for the following operations (5 Marks):

   a) Insertion at the beginning

   b) Insertion at the end

   c) Deletion from the beginning

   d) Display the linked list

4. Which linked list contains a link from the last node back to the first node? (1 Mark)

   A) Singly Linked List

   B) Doubly Linked List

   C) Circular Linked List

   D) Linear List

5. Analyze the time complexity of the following singly linked-list operations (5 Marks):

   a) Creating a linked list

   b) Traversing the list

   c) Searching for an element

   d) Inserting at the beginning

   e) Deleting a node

6. Write a C program to create a doubly linked list and perform forward and backward traversal. Evaluate the advantages of maintaining both prev and next pointers (5 Marks).

7. In a doubly linked list, each node normally contains: (5 Marks)

   A) Data and one pointer

   B) Data and two pointers

   C) Only two pointers

   D) Data and three pointers

8. What is a linked list? Explain the different types of linked lists with suitable diagrams (5 Marks).

STACK AND QUEUE

1. Which principle is used in the implementation of a stack? (1 Mark)

A) FIFO

B) LIFO

C) FCFS

D) Random Access

2. Which operation removes the topmost element from a stack? (1 Mark)

A) Push

B) Insert

C) Pop

D) Delete

3. Explain the concept of a circular queue with a neat, labelled diagram and describe how it differs from a linear queue (any one difference) (5 Marks).

4. Explain the working of a stack and describe its basic operations with a suitable example (5 Marks).

5. Explain the working of a queue and describe its basic operations with a suitable example (5 Marks).

6. Write a C program to implement Stack using an array with the following operations: PUSH, POP, DISPLAY. Evaluate your implementation by handling both Stack Overflow and Stack Underflow.

7. Write a C program to implement a Queue using an array with: INSERT, DELETE, DISPLAY. Evaluate the implementation after multiple insertions and deletions (5 Marks).

8. Using a stack, apply the procedure for converting the following infix expression into postfix notation (5 Marks each):

a) A + B × (C – D)

b) (A – B) / (C + D) × E

c) A + B × (C – D)

d) A × (B + C) – D / E

9. Analyze the following postfix expression using a stack. Show the stack contents after each operation and determine the final result (5 Marks each):

a) 8 4 2 + * 6 –

b) 7 3 2 * + 5 –

c) 9 5 2 + * 4 –

d) 9 5 2 + * 4 –

UNIT 5 – Trees, Graphs and Hashing

TREE

1. Which node in a tree has no parent?

A) Leaf Node

B) Root Node

C) Sibling Node

D) Internal Node

2. Explain the concept of tree traversal and describe its different types.

3. Construct a Binary Search Tree by inserting the following elements in the given order: 40, 20, 60, 10, 30, 50, 70. Apply the three standard tree traversal techniques and write the resulting sequences.

4. Apply the Binary Search Tree (BST) insertion algorithm to construct a BST using the following elements: 50, 30, 70, 20, 40, 60, 80. Then apply Inorder, Preorder, and Postorder traversals to the constructed tree.

GRAPH

1. In graph terminology, an edge that has a specified direction from one vertex to another is called:

A) Undirected Edge

B) Directed Edge

C) Adjacent Node

D) Successor Node

2. Analyze the differences between Adjacency Matrix and Adjacency List representations of a graph with suitable examples.

HASHING

1. Explain the basic concepts of hashing and graphs. Describe any one collision resolution technique in hashing.

2. Apply the hash function h(k) = k mod 10 to insert the following keys into a hash table of size 10 using linear probing: 23, 43, 13, 27, 37. Show the final hash table and indicate the positions where collisions occur.

3. Which hashing technique uses an additional hash function to determine the next position when a collision occurs?

A) Cuckoo Hashing

B) Tree Traversal

C) Adjacency Matrix

D) Linear Search